

Claude Code in VS Code

The tricks I use every day

A practical tour for friends: start, configure, plan, drive

Four blocks: **start, configure, plan, then drive daily**

1

Start

Claude's tools,
extension vs CLI,
opening a project

Slides 3-7

2

Configure

.claude/ folder,
CLAUDE.md, skills,
agents, hooks

Slides 8-14

3

Plan

Problem, decisions,
phases, exit gates,
"next phase"

Slides 15-17

4

Drive

Context tricks,
terminal flags, MCP,
review commands

Slides 18-24

1

Start

Tools, VS Code vs CLI, first run

In this section

What Claude can do

Extension vs CLI

Open a folder

Start the CLI

Claude works through tools: read, search, edit, run

Read

Opens files (code, images, PDFs)

Grep

Searches inside files for text or patterns

Glob

Finds files by name pattern

Edit

Changes an exact piece of an existing file

Write

Creates or fully replaces a file

Bash

Runs terminal commands: tests, git, uv

AskUserQuestion

Asks you to choose between options

Agent

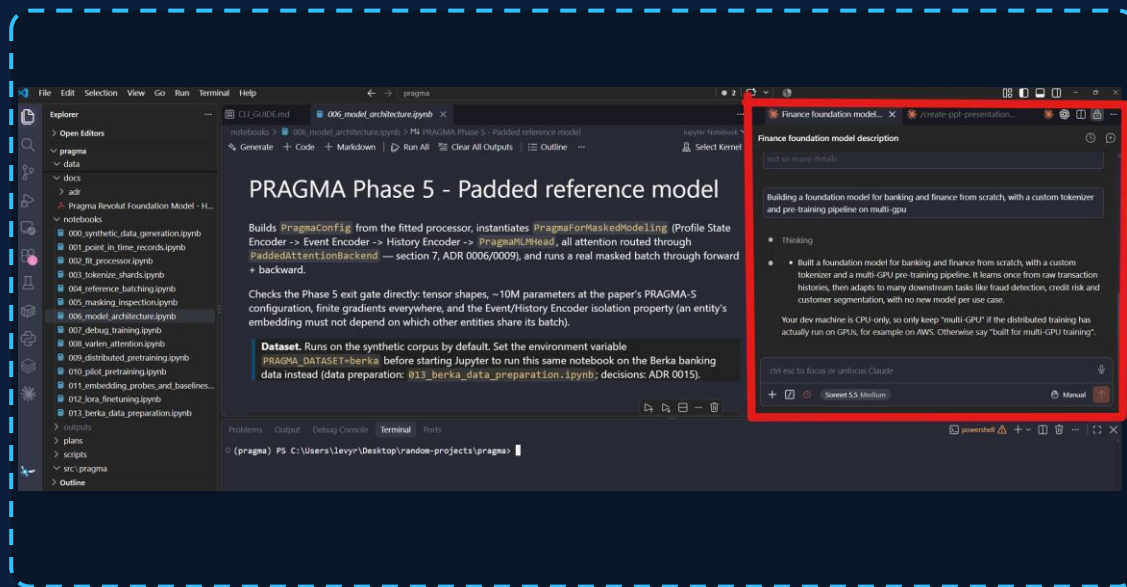
Hands a task to a sub-agent

Also: WebSearch, WebFetch, Skill... You decide which tools may run without asking (permissions).

Two front doors, same Claude: extension and CLI

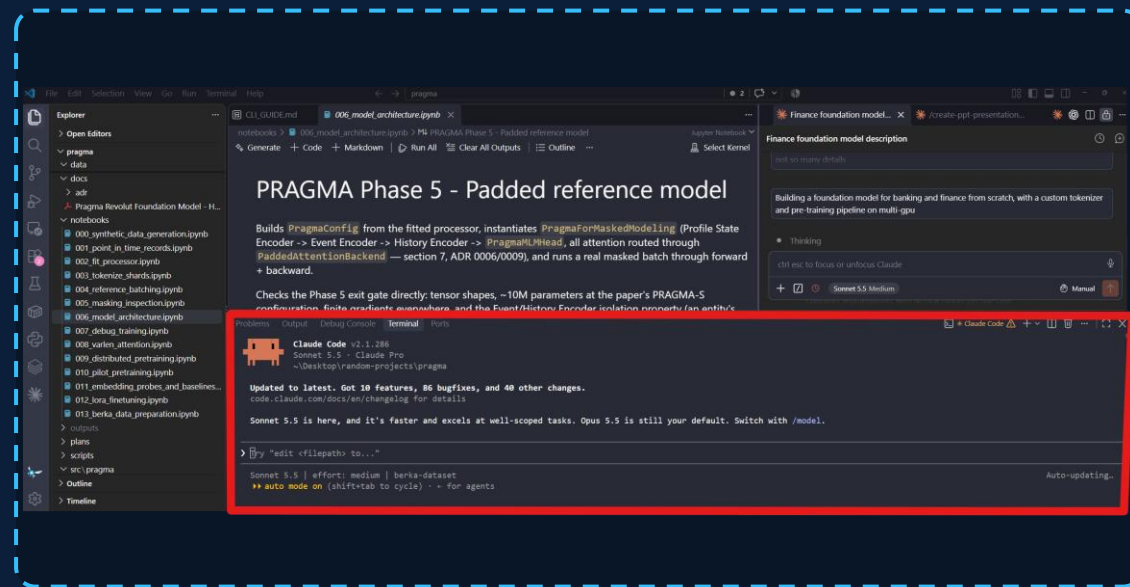
VS Code extension

Chat panel on the right side, inline diffs, @-mention files



CLI in a terminal

Run `claude` in any terminal, including the one inside VS Code



My split: extension for chat day-to-day, CLI for flags, agents and automation

Start in VS Code: open a folder, open the Claude panel

Step 1

File > Open Folder...

Claude only sees what is inside the folder.

Step 2

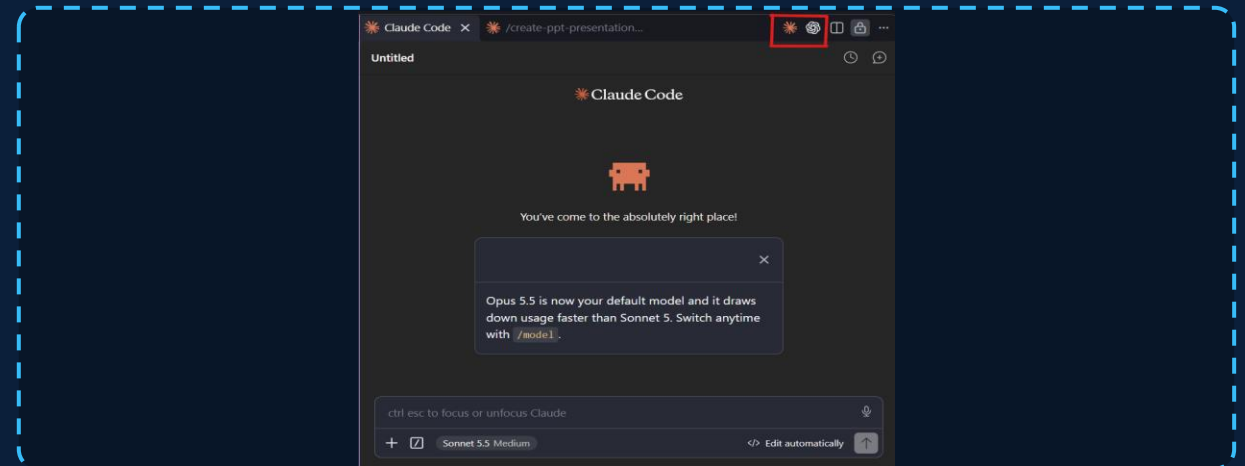
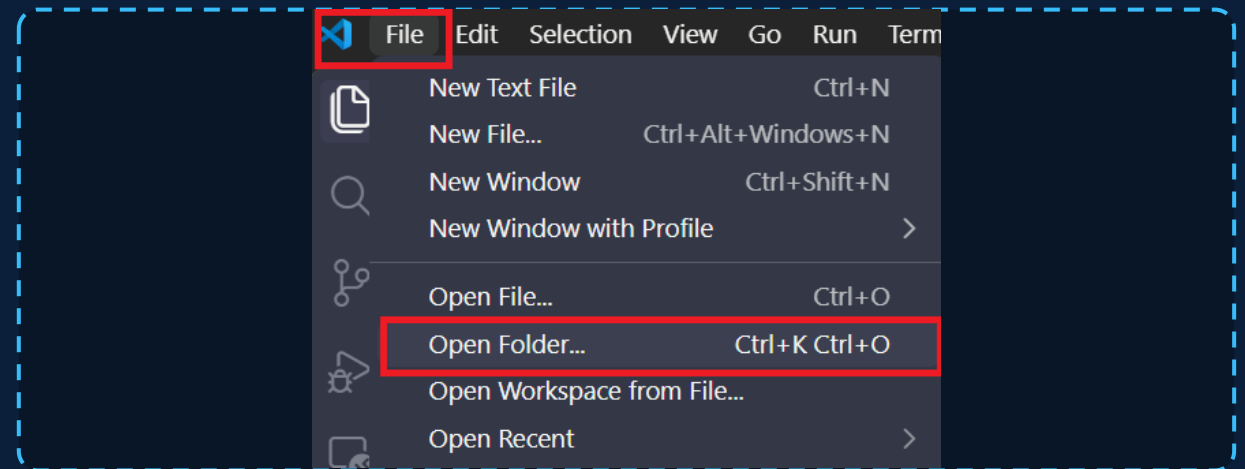
Open the Claude panel

Claude icon in the side menu (right side).

Step 3

Describe a task

Review diffs, then accept or push back.



Start the CLI: one word in any terminal

Three moves

- Open a terminal (VS Code: Ctrl+`)
 - Or go to 'Terminal' → 'New Terminal'
- Open folder
 - Or cd into your project folder
- Type `claude` and press Enter
- Type `/` to see every slash command

The image contains two screenshots of a VS Code terminal window. The top screenshot shows the terminal with the prompt `(pragma) PS C:\Users\levyr\Desktop\random-projects\pragma>` and the word `claude` entered at the end of the line. The bottom screenshot shows the terminal with the prompt `> /` and a list of slash commands displayed below it: `/pptx`, `/create-ppt-presentation`, and `/statusline`. To the right of these commands, there is a description: "Use this skill any time a .pptx decks, pitch decks, or presenta Plan and create a PowerPoint (. tech-executive, tech-heavy, com Set up Claude Code's status lin". In both screenshots, the "Terminal" tab in the VS Code interface is highlighted with a red box.

```
Problems Output Debug Console Terminal Ports
(pragma) PS C:\Users\levyr\Desktop\random-projects\pragma> claude

Problems Output Debug Console Terminal Ports
> /
  /pptx
  /create-ppt-presentation
  /statusline
Use this skill any time a .pptx
decks, pitch decks, or presenta
Plan and create a PowerPoint (.
tech-executive, tech-heavy, com
Set up Claude Code's status lin
```

2

Configure

Shape how Claude behaves in your project

In this section

.claude/ folder

CLAUDE.md

Skills, rules, hooks

My skills

Agents

Hooks and settings

The **.claude/ folder** is where you shape Claude's behavior

```
.claude/  
  settings.json  
  settings.local.json  
  agents/  
    summarizer.md  
  skills/  
    my-skill/SKILL.md  
  rules/  
    style.md  
  commands/  
  CLAUDE.md (project root)
```

<code>settings.json</code>	Permissions, hooks, env. Shared via git
<code>settings.local.json</code>	Your personal overrides, not committed
<code>agents/</code>	Sub-agents, one markdown file each
<code>skills/</code>	Repeatable playbooks, one folder each
<code>rules/</code>	Small focused instruction files
<code>commands/</code>	Your own slash commands

Same layout in `~/.claude/` applies to all your projects

CLAUDE.md: Claude's briefing note for your project

What goes in it

- What the project is and its layout
- Commands: how to run, test, lint
- Conventions and style preferences
- Where the plans and decisions live
- Do's and don'ts (e.g. never commit unasked)

Why it's worth having

- Loaded every session: no re-explaining
- Shared with the team through git
- Fewer wrong guesses, fewer corrections
- A global one in ~/.claude/ holds personal habits

```
> Create a CLAUDE.md for this project: what it is, how to run and test it,  
and the conventions you see in the code. (or just type /init)
```

Skills, rules, hooks, agents: pick the right block

Repeatable behavior

Skills

A playbook Claude loads when the task matches, or when you type /name.

Isolated behavior

Rules

Small focused instruction files, optionally scoped to certain files.

Mandatory behavior

Hooks

Commands that run automatically on events. Guaranteed, not suggested.

Delegated work

Agents

Specialist helpers with their own context, tools and model.

CLAUDE.md = always-on context. These four = on-demand or automatic behavior.

Example of my skills: Claude wrote them, and I keep asking it to edit them

Skill	What it does
create-new-project	Scaffolds a Python uv project with plans/, notebooks/, src/
create-notebook	Builds local or Kaggle Jupyter notebooks
create-tests / run-tests	Writes tests; runs pytest + ruff + mypy like CI
create-cli	Adds a small CLI plus a CLI.md on how to run it
create-ppt-presentation	Plans and builds a .pptx deck (like this one)
create-video	Narrated explainer video from a script (MP4)
git-flows	Checks GitHub identity, branch and commit conventions

Ask: "Create a skill that does X every time I ask for Y". Later: "Edit that skill so it also Z".

Agents: a markdown file in `.claude/agents/`

```
---
name: summarizer
description: Summarize files in 5
bullets
tools: Read, Grep, Glob
model: haiku
---
You read the given files and
return a 5-bullet summary.
```

Ask Claude to write it, or use `/agents`

Built-in agents

- Explore: fast read-only search
- Plan: designs an approach
- General-purpose: multi-step work

How to call them

- `@agent-summarizer` in the chat
- `claude --agent` coordinator for the whole session
- Or just ask: Claude delegates by itself

Hooks and settings: get a push notification when Claude finishes

```
{
  "hooks": {
    "Stop": [{
      "hooks": [{
        "type": "command",
        "command": "<your notify script>"
      }]
    }]
  }
}
```

Events: Stop, Notification, PreToolUse...

Settings levels

- ~/.claude/settings.json: all projects
- .claude/settings.json: this project, shared
- .claude/settings.local.json: just you

Easiest way

- Ask: "Add a hook that notifies me when you finish"
- Use /hooks to see what's active



3

Plan

A process that scales beyond one prompt

In this section

Problem and decisions

Plans, tests, gates

Phases

Saying "next phase"

Plan first, then build in phases: my process

1

Problem

What am I solving,
for whom?

2

Decisions

Settle open
choices before
coding

3

Detailed plan

Written to
plans/001-plan-
name.md

4

Tests + criteria

Acceptance
criteria, test plan

5

Phases + gates

Exit gate to pass
before the next

Exit gate example: "tests pass, ruff and mypy clean, debug run works"

A written plan survives context resets. Claude re-reads it, so nothing is lost.

Then the loop is one sentence: **next phase**

```
plans/  
  001-plan-main.md  
    Phase 1: build, tests  
    Exit gate: tests pass  
    Phase 2: ...  
progress.md  
  Status: phase 2 done  
  Done / In progress / Next  
  Gate evidence
```

1

Claude builds the phase

2

Runs tests and the exit gate

3

Updates progress.md with evidence

4

I review, then say: next phase

4

Drive

Daily commands and flags

In this section

Context management

Terminal flags

MCP

Review commands

Remote and learning

Habits

/clear often: keep the context lean

/context

See what fills the context window

/clear

Fresh start between unrelated tasks

/rewind

Roll the conversation back to an earlier point

/memory

View and edit what Claude remembers

/btw

Side question that doesn't clutter the chat

/rename

Name a session so you can resume it later

/goal

Set a goal Claude keeps working toward

Works in the extension chat and in the CLI

Terminal flags: resume, fork, script, and run in parallel

```
claude -c
```

Continue the most recent conversation

```
claude --resume
```

Pick an earlier session to resume

```
--fork-session
```

Branch off a session, keep the original

```
claude -p "..."
```

One prompt, plain output, for scripts

```
claude --agent  
coordinator
```

Run the whole session as that agent

```
claude --worktree add-  
billing
```

Isolated git worktree for parallel work

MCP: plug outside tools into Claude

What it is

- A standard way to **connect Claude to external tools and data**
- Add it once, Claude can then use it in any chat
- In chat, /mcp shows connected servers
- Only add servers you trust

```
# list configured servers
claude mcp list

# add a remote (HTTP) server
claude mcp add --transport http <name> <url>
```

Add --scope user to make it available in all projects

Check and ship with the **built-in commands**

/run

Launch the app and see the change working

/verify

Check that the work actually does what was asked

/code-review

Review the diff for bugs; "ultra" runs a deep cloud review

/security-review

Security check of the pending changes

/simplify

Clean up changed code for reuse and clarity

/install-github-app

Connect Claude to your GitHub repo and PRs

Take it with you, and keep learning inside the tool

Anywhere

/remote-control

Continue a running session from another device. Start a long task, **then check on it from your phone.**

Learn

/powerup

Interactive in-product lessons for features you haven't tried yet. A good way to find new tricks.

Eight habits to take home

1

Open a folder

File > Open Folder..., then
start Claude

2

CLAUDE.md

Ask Claude to write it

3

Plan before code

Plans, tests, exit gates

4

Phases

Say "next phase"

5

Keep context lean

/clear, /context, /rewind

6

Config via Claude

Skills, agents, hooks

7

Review

/code-review, /security-
review

8

Parallelize

--worktree, agents